

FOOTBALL LEAGUE SIMULATOR - FULL TECHNICAL DOCUMENTATION

Bu doküman, Senior Software Engineer ve Technical Mentor perspektifinden hazırlanmış, projeyi A'dan Z'ye anlatan, her kararı savunan ve mülakatlara hazırlayan tam kapsamlı teknik analiz dokümanıdır.

İÇİNDEKİLER

- [Project Overview](#)
- [Folder Structure Analysis](#)
- [GoLang Concepts Used](#)
- [Design Patterns](#)
- [System Architecture & Data Flow](#)
- [File-by-File & Line-by-Line Explanation](#)
- [Database Design & SQL Query Analysis](#)
- [Algorithms](#)
- [Endpoint Architecture](#)
- [Interview Defense Preparation](#)
- [Technical Debt, Scaling & Improvement Areas](#)

1. PROJECT OVERVIEW

Bu proje, Premier League kurallarına uygun olarak çalışan 4 takımlı bir futbol ligi simülatörüdür.

- Dil:** Go (Golang)
- Veritabanı:** SQLite
- Mimarî:** Layered Architecture (Katmanlı Mimari)
- Ana Hedef:** Kullanıcıya REST API üzerinden hafta hafta maç oynatmak, puan tablosunu hesaplamak ve sezonun sonuna doğru (4. haftadan sonra) şampiyonluk tahmini sunmak.

2. FOLDER STRUCTURE ANALYSIS

Projenin klasör yapısı "Standard Go Project Layout" standartlarına uygun tasarlanmıştır.

```
football-league-simulator/  
├── cmd/  
│   └── server/  
│       └── main.go  
├── internal/  
│   ├── db/  
│   ├── handlers/  
│   ├── models/  
│   └── service/  
├── schema/  
│   ├── queries.sql  
│   └── schema.sql  
├── go.mod  
└── go.sum
```



cmd/

- **Amaç:** Uygulamanın giriş noktalarını (entry points) barındırır.
- **Neden gerekli:** Bir projede birden fazla çalıştırılabilir uygulama olabilir (örn: API sunucusu, background worker, CLI tool). Her biri `cmd/` altında kendi klasörüne sahip olur.
- **Silinirse ne olur:** Uygulamayı derleyecek veya başlatacak `main` paketi bulunamaz.
- **Industry Standard:** Go'da de facto standarttır.



internal/

- **Amaç:** Projeye özel, dışarıya kapalı olan (private) kodların bulunduğu yerdir.
- **Neden internal:** Go derleyicisi (compiler), `internal/` klasöründeki paketlerin başka projeler tarafından `import` edilmesini fiziksel olarak engeller. Bu bir kapsülleme (encapsulation) kuralıdır.
- **Mülakat Sorusu:** "Neden kodları root dizine değil de internal'a koydun?"
 - *Cevap:* "Domain logici dışarıdan izole etmek ve encapsulation sağlamak için. Bu bir Go idiomu. Yarın bu repoyu public yaparsam, kimsenin benim service mantığımı kendi projesine import edip bağımlılık yaratmasını istemem."



internal/handlers/

- **Amaç:** HTTP isteklerini (Request) karşılayıp, HTTP yanıtlarını (Response) oluşturur. JSON parse işlemleri ve HTTP status kodları burada yönetilir. Business logic İÇERMEZ.



internal/service/

- **Amaç:** Uygulamanın kalbidir. Business logic (iş mantığı) buradadır. Simülasyon, algoritma, tahminleme burada yapılır. Handler'dan veri alır, işler, sonucu döner.



internal/models/

- **Amaç:** Uygulama genelinde kullanılan veri yapılarını (Structs) tanımlar.



internal/db/

- **Amaç:** Veritabanı bağlantısının (Connection Pool) kurulması ve şema (schema) işlemlerinin yapıldığı klasördür.

3. GOLANG CONCEPTS USED

Struct Composition (Embedding)

Go'da sınıflar (classes) ve kalıtım (inheritance) YOKTUR. Bunun yerine Composition (Birleştirme) vardır.

```
type Team struct {
    ID      int
    Name    string
    Strength int
    Stats   // <-- Embedding (Composition)
}
```

Neden kullanıldı: "Has-a" (Sahiptir) ilişkisi kurmak için. `Team` objesi üzerinden direkt olarak `team.Points` veya `team.GF` şeklinde `Stats` struct'ının alanlarına erişebiliriz. Memory allocation açısından da `Stats` struct'ı pointer değil, by-value gömüldüğü için veri ardışık (contiguous) memory bloklarında tutulur, bu da CPU Cache hit oranını artırır.

Interface-Based Design

```
type LeagueManager interface {  
    PlayNextWeek() ([]models.Match, error)  
    // ...  
}
```

Neden kullanıldı: `handlers` paketi gerçek bir `leagueManager` struct'ı beklemez, sadece bu interface'i bekler. Bu sayede Dependency Injection yapabiliriz. Birim test (Unit Test) yazarken veritabanına bağlanmayan sahte (Mock) bir service enjekte edebiliriz. Bu, SOLID'in Dependency Inversion prensibidir.

4. DESIGN PATTERNS

Dependency Injection (DI)

`main.go` dosyasında veritabanı bağlantısı oluşturulur, `service` katmanına verilir. Service oluşturulur, `handler` katmanına verilir. Objeler kendi bağımlılıklarını kendileri yaratmaz (örn: Handler içinde `db.Connect` denmez), dışarıdan parametre olarak alırlar.

Service Layer Pattern

Veritabanı işlemleri (SQL sorguları) ve HTTP işlemleri (JSON) arasındaki iş mantığı (Maç simülasyonu, puan hesaplama) tamamen izole edilmiş bir Service katmanına alınmıştır.

5. SYSTEM ARCHITECTURE & DATA FLOW

Mimari Çizim (ASCII)

```
[CLIENT (Postman/Curl)]  
  |  
  ▼ (HTTP JSON Request)  
[ROUTER (chi)]  
  |  
  ▼ (Route Matching)  
[HANDLER (internal/handlers)]  
  |  
  ▼ (Method Call on Interface)  
[SERVICE (internal/service)]  
  |  
  ▼ (SQL Queries + Business Logic + Algoritma)  
[DATABASE (SQLite)]
```

Data Flow Analizi (POST /api/simulate-week)

- Request:** Kullanıcı `/api/simulate-week` 'e POST isteği atar.
- Router:** `chi` router bunu yakalar ve `h.SimulateWeek` handler fonksiyonuna yönlendirir.
- Service Çağırışı:** Handler, `h.svc.PlayNextWeek()` fonksiyonunu çağırır.
- Current Week Check:** Service, veritabanına gidip oynanmamış en küçük haftayı bulur (`currentWeek`).

5. **Fetch Data:** O haftanın maçları ve takımların mevcut güç/puan durumları DB'den çekilir. Memory'de bir map (`teamMap`) oluşturulur.
6. **Transaction Başlar:** Veri bütünlüğü için `tx, err := m.db.Begin()` ile işlem açılır.
7. **Simulation Loop:**
 - o `simulate()` çağrılır, goller hesaplanır.
 - o `home.Apply()` ve `away.Apply()` ile takımların statsları RAM üzerinde güncellenir.
 - o Oynanan maçlar ve takım güncellemeleri DB'ye UPDATE sorguları ile gönderilir.
8. **Commit:** Hata yoksa `tx.Commit()` yapılarak DB kalıcı hale getirilir.
9. **Response:** İşlenmiş maçlar Handler'a döner, Handler bunu JSON'a çevirip Client'a `200 OK` olarak yollar.

6. FILE-BY-FILE & LINE-BY-LINE EXPLANATION

`internal/models/models.go`

Genel Amaç: Projedeki tüm entity'leri barındırır. Sadece veri tutar, dış bağımlılığı yoktur.

```
type Stats struct { ... } // Lig puan cetveli kolonları
func (s *Stats) Apply(goalsFor, goalsAgainst int) // Maç sonucunu ekler
func (s *Stats) Revert(goalsFor, goalsAgainst int) // Maç sonucunu geri alır
```

- **Neden Receiver Methods?** Go'da metotlar structlara bağlanır. `s *Stats` bir pointer receiver'dır. Kopyası üzerinde değil, doğrudan orijinal memory adresindeki veriyi değiştirmek için pointer kullanılır.

`cmd/server/main.go`

Genel Amaç: Wiring (Kablolama) işlemlerini yapar. Uygulamayı ayağa kaldırır.

```
func main() {
    schemaPath := resolveSchemaPath()
    database, err := db.Open(dbPath, schemaPath)
    leagueService, err := service.NewLeagueManager(database)
    handler := handlers.New(leagueService)
    http.ListenAndServe(addr, handler.Routes())
}
```

- **Adım Adım:** Schema path çözülüyor -> DB bağlantısı açılıyor -> DB kullanılarak Service yaratılıyor -> Service kullanılarak Handler yaratılıyor -> Server 8080 portunda dinlemeye başlıyor.

`internal/service/service.go`

Genel Amaç: Tüm kuralların, hesaplamaların olduğu kalptir. Dev bir dosyadır.

```
func (m *leagueManager) playWeek(week int) ([]models.Match, error) {
    fixtures, _ := m.fetchUnplayedWeekMatches(week)
    teamMap, _ := m.fetchAllTeamsMap()
    tx, _ := m.db.Begin()
    defer tx.Rollback()
```

```
// ...  
}
```

- **Neden `tx.Rollback()` defer edildi?** `defer` fonksiyon bitmeden hemen önce çalışır. Eğer işlem başarılıysa biz zaten sonda `tx.Commit()` diyoruz, commitlenmiş bir tx rollback edilemez (hata vermez, ignorlanır). Ama kodun ortasında bir hata olursa fonksiyon erken döner (`return err`) ve `defer` çalışarak işlemleri geri alır (Atomic işlem garantisi).

7. DATABASE DESIGN & SQL QUERY ANALYSIS

Veritabanı Seçimi: SQLite

Neden: Setup gerektirmez, dosya bazlıdır, prototype/case study için idealdir. (Mülakat cevabı: "Production'da Postgres kullanırdım ancak case sürecinde hızlı ayağa kalkması ve review edecek kişinin kurulumla uğraşmaması için SQLite tercih ettim.")

Önemli Go Ayarı: `database.SetMaxOpenConns(1)`

- **Neden:** SQLite concurrent (eşzamanlı) yazma işlemlerini desteklemez. Go'nun `database/sql` paketi default olarak connection pool kullanır. Eğer iki goroutine aynı anda db'ye yazmaya kalkarsa "database is locked" hatası fırlatır. Bunu engellemek için pool limitini 1'e çektik.

Şema:

`teams` tablosu: id, name, strength, won, drawn, lost, gf, ga, gd, points. `matches` tablosu: id, week, home_id, away_id, home_score, away_score, played (boolean olarak 0/1).

- **Relational Integrity:** `home_id` REFERENCES `teams(id)` foreign key constraintleri kullanıldı.

Örnek Sorgu Analizi (Puan Durumu):

```
SELECT id, name, strength, played, won, drawn, lost, gf, ga, gd, points  
FROM teams  
ORDER BY points DESC, gd DESC, gf DESC, name ASC;
```

- **Mantık:** Premier League kuralları sırasıyla: Puan (points), Averaj (gd - goal difference), Atılan Gol (gf - goals for) ve isme göre alfabetik sıralar.

8. ALGORITHMS

1. Match Simulation Algorithm (Maç Simülasyonu)

```
func (m *leagueManager) simulate(home, away models.Team) (int, int) {  
    ha := float64(home.Strength+homeStrengthBonus) / 100.0  
    aa := float64(away.Strength) / 100.0  
    return m.goals(ha), m.goals(aa)  
}  
  
func (m *leagueManager) goals(attack float64) int {  
    n := 0  
    for i := 0; i < 5; i++ {  
        if m.rng.Float64() < attack { n++ }  
    }  
    return n  
}
```

```
}  
    return n  
}
```

- **Mantık:** Bernoulli Trial (Bernoulli deneyi). Her takım 5 kez atak yapar (`attackRounds`). Her atakta gol olma ihtimali takımın gücüne eşittir.
- **Örnek:** Manchester City'nin gücü 90. Ev sahibi olduğu için $+5 = 95$. Her atakta %95 ihtimalle gol atar. Random sayı (0.00-1.00 arası) 0.95'ten küçükse gol olur.
- **Neden bu algoritma?** Futbol şans oyunudur. 95 gücündeki takım 5 atakta hiç gol atamayabilir (düşük ihtimal olsa da). Bu, lige sürpriz elementleri (underdog wins) katar.

2. Championship Prediction Algorithm (Şampiyonluk Tahmini)

```
score = (Points * 10) + (GD * 2) + (Strength * 1)
```

- **Mantık:** Deterministik, ağırlıklı skor modeli (Weighted Score Model).
- **Time Complexity:** $O(N)$ -> N =Takım sayısı.
- Puan en önemli metrik olduğu için x10. Averaj resmi tiebreaker olduğu için x2. Takım gücü kalan haftalardaki kazanma potansiyeli olduğu için x1. Bulunan skorlar toplanıp takımlara yüzde (Percentage) olarak dağıtılır ($Score / TotalScore * 100$).

3. Fixture Generation Algorithm (Berger Round-Robin)

- Çift devreli (Double round-robin) lig algoritması. Bir takım sabit kalırken diğerleri etrafında döner (daireysel shift). Böylece her takım birbiriyle bir kez içerde, bir kez dışarıda oynar. Time Complexity: $O(N^2)$.

9. ENDPOINT ARCHITECTURE

RESTful prensiplere göre tasarlandı. Router olarak `chi` kullanıldı. Chi çok hızlı, lightweight ve net/http standartlarına tamamen uygun bir router'dır.

- `GET /api/standings` : Lig tablosunu getirir.
- `GET /api/matches` : Tüm oynanmış ve oynanacak maçların listesi.
- `GET /api/week/{n}` : Belirli bir haftanın özetini (Tablo + Maçlar + Tahmin) döner.
- `POST /api/simulate-week` : Gelecek ilk unplayed haftayı oynatır. (Neden POST? Çünkü sistemin state'ini değiştiriyor).
- `POST /api/simulate-all` : Kalan tüm haftaları tek seferde oynatır.
- `PUT /api/match/{id}` : Belirli bir maçın sonucunu değiştirir. (Idempotent olduğu için PUT kullanıldı).

10. INTERVIEW DEFENSE PREPARATION

Mülakatta Gelebilecek Sorular ve "Vurucu" Cevaplar:

1. **Soru:** Neden Interface kullandın? Doğrudan struct kullanabilirdin.
 - **Cevap:** "Handler katmanının Service katmanına sıkı sıkıya bağlanmasını (tight-coupling) istemedim. Interface kullanarak Inversion of Control sağladım. Yarın birisi benden bu kodlara Unit Test yazmamı isterse, veritabanına giden gerçek service yerine, bu interface'i implement eden mock bir obje yaratıp handler'larımı saniyeler içinde test edebilirim."

2. **Soru:** Transaction'ları nasıl yönetiyorsun? Neden gerek duydun?

- o **Cevap:** "Bir hafta simüle edildiğinde 4 takımın istatistiği ve 2 maçın skoru güncelleniyor. Yani 6 ayrı SQL update sorgusu atılıyor. Eğer 3. sorguda veritabanı çökerse veya hata olursa, tablo bozulur (Inconsistent state). Bu yüzden `db.Begin()` ile işlem başlatıp, tüm güncellemeler bittikten sonra `Commit()` ediyorum. Kod ortasında panic veya error olursa `defer tx.Rollback()` devreye giriyor. Atomicity (ACID) sağladım."

3. **Soru:** Prediction (Tahmin) sistemini nasıl modelledin? Machine Learning kullanılamaz mıydı?

- o **Cevap:** "4 takımlı ve kısıtlı verili bir sistemde ML overkill (gereksiz karmaşık) olurdu. Monte Carlo simülasyonu yapıp kalan maçları 10.000 kez oynatıp çıkan sonuçların ortalamasını alabilirdim (bunu değerlendirdim). Ancak performanslı ve deterministik olması için ağırlıklı skor (Weighted Score) algoritması yazdım. Puan, Averaj ve Takım Gücüne farklı katsayılar vererek $O(N)$ hızında çalışan bir sistem tasarladım."

4. **Soru:** Neden Golang seçtin?

- o **Cevap:** "Go, statik tipli, compile edilen, concurrency desteği muazzam ve boilerplate (gereksiz kalıp) kod yazımını engelleyen bir dil. Case study için en hızlı ayağa kalkan, tek bir binary file üreten, dependency'si çok az olan bir mimari sunuyor."

5. **Soru:** Edit match result (maç düzenleme) kısmındaki Revert ve Apply mantığı nedir?

- o **Cevap:** "Bir maçın sonucunu 2-1'den 3-0'a çevirdiğinizde, puan tablosunu baştan hesaplamak yerine, eski sonucun (2-1) takımlara verdiği puan ve averajı siliyorum (Revert), ardından yeni sonucu (3-0) takımların istatistiklerine uyguluyorum (Apply). Bu $O(1)$ hızında çalışıyor ve mimariyi muazzam derecede modüler tutuyor."

11. TECHNICAL DEBT, SCALING & IMPROVEMENT AREAS

Bir Senior olarak, kendi yazdığım kodun eksiklerini ve "Production'da olsaydı neleri farklı yapardım" kısmını bilmek çok önemlidir.

Weaknesses (Zayıf Yönler)

- **Repository Pattern Eksikliği:** Şu an `service.go` dosyası doğrudan SQL sorgularına bağımlı. Klasik mimarilerde `internal/repository/` klasörü açılıp, SQL sorguları oraya taşınmalı, `service` sadece `repository interface`'i ile konuşmalıdır.
 - o *Neden böyle yapıldı?* Prototyping hızlandırmak ve over-engineering'den (aşırı mühendislikten) kaçınmak için.
- **In-Memory Loading:** `simulateWeek` fonksiyonunda takımları ID'ye göre kolay çekebilmek için `fetchAllTeamsMap()` ile tüm takımları RAM'e alıyorum. 4 takım için harika. Ancak 10.000 takım olsa RAM dolardı.

Production Readiness (Enterprise Seviyeye Çıkarmak İçin Gerekenler)

1. **Veritabanı Değişikliği:** SQLite sadece bir connection kabul ediyor (`MaxOpenConns=1`). Bu yatayda ölçeklenemez (Horizontal Scaling). Production'da PostgreSQL'e geçilmeli.
2. **Dockerization:** `Dockerfile` ve `docker-compose.yml` yazarak uygulamanın ve veritabanının containerize edilmesi gerekir.
3. **Environment Variables:** `dbPath` ve `port` gibi değerler kod içinde hardcoded (sabit). Bunlar `viper` veya `godotenv` kütüphaneleri ile `.env` dosyasından okunmalıdır.

4. **Unit Tests:** `service_test.go` ve `handlers_test.go` yazılarak `test coverage` %80'in üzerine çıkarılmalıdır.

5. **Observability:** Prometheus metrikleri ve Jaeger/OpenTelemetry ile tracing eklenmelidir.

Final Summary

Bu proje; Interface-Based Design'ı anlayan, struct embedding'in farkında olan, transaction yönetimini (ACID) doğru kurgulayan ve veritabanı kilitlenmelerini (locking) öngörüp ona göre design pattern belirleyen bir yazılımcının elinden çıkmıştır. Gereksiz framework'lerden kaçınılmış, Go'nun sadeliği ön planda tutulmuştur.